

RlayHub

Blokzincir Geliştiriciliği Katılımcı Kitabı

Dört günlük kampın tamamı, kurulumdan Demo Day'e.
Kampta anlatılan her şeyin yazılı hali.

Nesin Matematik Köyü, Şirince

Ağustos 2026

Eğitmen: Meriç Cintosun

Ağ: Base Sepolia testnet (zincir kimliği 84532)

İçindekiler

0. Başlamadan Önce: Bilgisayarını Hazırla

Neden terminal
Windows kullanıyorsan: WSL
Git ve GitHub
Node, npm, nvm
Foundry: forge, cast, anvil
Cüzdan
Editör ve eklentiler
OpenZeppelin ve kütüphaneler
İşine yarayacak diğer araçlar
Kurulum kontrol listesi

1. Felsefe: Bu Şey Neden Var

Merkeziyetsizlik neden doğdu
Satoshi neden anonim kaldı
Kuruma değil koda güvenmek
Gereksiz blokszincir

2. Temeller: Defter, Mühür, Mutabakat

Ortak defter fikri
Çift harcama
Mühür ve değiştirilemezlik
Cüzdan, adres, özel anahtar
Bir işlemin yolculuğu
Ücret neden var, L2 neden ucuz

3. Akıllı Kontratlar

Otomat benzetmesi
Üç parça: hafıza, işlem, duyuru
msg.sender ve kural yazmak
Custom error, modifier, sahiplik
ERC-20 ve neden sıfırdan yazmıyoruz
Deploy ve kaynak kodu doğrulama

4. AI ile Kod Yazmak

Prompt'un dört parçası: bağlam, rol, içerik, hedef
AI ile kod yazmanın beş kuralı
Şartname, test, kod döngüsü
AI'ın yapamadıkları
Bug'ı bulmak: yeşil test yalan söyler

5. Güvenlik

Reentrancy: bir kasa nasıl boşaltılır

Önce kontrol et, sonra kaydet, en son gönder

Üç gerçek vaka kalıbı

Yeni başlayanın kontrol listesi

6. Arayüz: Zinciri Ekranı Bağlamak

Okuma ve yazma farkı

wagmi ve viem

Bekleme ve hata durumu

7. Hackathon

Üç dakikada sunum

Jüri neye bakar

En sık yapılan sekiz hata

Kampın ardından nasıl çalışılır

8. Yazılımcı ve Geliştirici

İki farklı kavram

AI yazmayı ucuzlattığında geriye ne kalıyor

Builder olmak

Ekler

Komut referansı

Sözlük

Kaynaklar

BÖLÜM 0

Başlamadan Önce: Bilgisayarını Hazırla

Kampın ilk günü bilgisayarına hiçbir şey kurmuyorsun, sadece tarayıcı yetiyor. İkinci günün öğleden sonrasından itibaren terminal açılıyor. Bu bölüm o ana kadar hazır olman gereken her şeyi anlatıyor. Kampa gelmeden kurabilirsen bir öğleden sonra kazanırsın.

0.1 Neden terminal

Blokszincir araçlarının neredeyse tamamı komut satırında çalışıyor. Bunun sebebi araçların eski olması değil. Yaptığın işin tekrarlanabilir olması gerekiyor. Bir butona tıkladığında ne olduğunu kimse göremez, ama bir komutu yazdığında o komutu başkasına gönderebilirsin, bir dosyaya kaydedebilirsin, bir sonraki sefer aynısını çalıştırabilirsin.

Terminal korkutucu görünüyorsa şunu bil: kampta kullanacağın komut sayısı yirmiyi geçmiyor. Hepsi bu kitabın sonundaki referansta var.

0.2 Windows kullanıyorsan: WSL

Bu maddeyi atlama. Windows kullanıcılarının kampta yaşadığı sorunların çoğu buradan çıkıyor.

Blokszincir araçlarının büyük kısmı Linux ve macOS için yazılıyor. Windows'un kendi komut satırı (cmd, PowerShell) dosya yollarını, izinleri ve satır sonlarını farklı ele alıyor. Foundry gibi araçlar Windows'ta ya hiç kurulmuyor ya da yarım çalışıyor.

Çözüm WSL. Açılımı Windows Subsystem for Linux. Windows'un içinde çalışan gerçek bir Linux. Sanal makine değil, ayrı bilgisayar değil, aynı dosyalara erişebiliyorsun ve neredeyse hiç yavaşlama yok.

Kurulum

PowerShell'i yönetici olarak aç, şunu yaz:

```
wsl --install
```

Bilgisayarı yeniden başlat. Açılışta Ubuntu bir kullanıcı adı ve parola soracak. Bu parola Windows parolan değil, Linux tarafının parolası. Not al, çünkü her sudo komutunda soracak.

Bundan sonra terminal derken Ubuntu terminalini kastediyoruz. PowerShell'i kapat, orada çalışma.

SIK YAPILAN HATA

Projeyi Windows tarafında (C:\Users\...) tutup WSL'den erişmek. Çalışır ama çok yavaştır ve dosya izinleri karışır. Projeyi Linux tarafında tut. WSL terminalinde `cd ~` yazdığında gittiğin yer orası.

macOS veya Linux kullanıyorsan bu bölümü atla, senin terminalin zaten hazır.

0.3 Git ve GitHub

Git bir sürüm kontrol aracı. Yaptığın her değişikliğin fotoğrafını çekiyor, istediğin ana geri dönebiliyorsun. GitHub ise o fotoğrafları sakladığın ve başkalarıyla paylaştığın site. İkisi aynı şey değil. Git bilgisayarında çalışır, GitHub internette durur.

Kurulum

```
sudo apt update
sudo apt install git      # WSL / Ubuntu
brew install git          # macOS
git --version
```

İlk ayar

Git her kaydın kim tarafından yapıldığını yazıyor. Bir kere ayarla:

```
git config --global user.name "Adin Soyadin"
git config --global user.email "eposta@ornek.com"
```

Kampta kullanacağın komutlar

```
git clone <adres>      # bir projeyi bilgisayarına indir
git status              # ne degistirdim
git diff                # tam olarak neyi degistirdim
git add .               # degisiklikleri kayda hazirla
git commit -m "mesaj"  # kaydi al
git push                # GitHub'a gonder
git pull                # GitHub'daki degisiklikleri al
```

Kampta en çok işine yarayacak olan `git diff`. Üçüncü gün yapay zekaya kod yazdıracaksın ve neyi değiştirdiğini görmek gerekecek. O anda bu komut senin gözün olacak.

GitHub hesabı

Ücretsiz aç. SSH anahtarı kurmanı öneririm, her seferinde parola sormaz:

```
ssh-keygen -t ed25519 -C "eposta@ornek.com"
cat ~/.ssh/id_ed25519.pub
```

Çıkan metni GitHub'da Settings, SSH and GPG keys bölümüne yapıştır. Test et:

```
ssh -T git@github.com
```

0.4 Node, npm, nvm

Üç isim karışıyor, ayıralım.

İsim	Nedir
Node.js	JavaScript'i tarayıcı dışında çalıştıran program. Arayüz tarafının motoru.
npm	Node ile birlikte gelen paket yöneticisi. Başkalarının yazdığı kütüphaneleri indirir.
nvm	Node'un sürümlerini yöneten araç. Aynı bilgisayarda birden fazla Node sürümü tutmanı sağlar.

Node'u doğrudan kurma, nvm ile kur. Sebebi şu: farklı projeler farklı Node sürümü istiyor ve doğrudan kurulmuş Node'u değiştirmek zahmetli.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash
```

Terminali kapat aç. Sonra:

```
nvm install --lts
nvm use --lts
node --version    # v20.9 veya üstü olmalı
npm --version
```

Kampın arayüz projesi Node 20.9 ve üstünü istiyor. Daha eskisi varsa Next.js açılmaz.

0.5 Foundry: forge, cast, anvil

Foundry, akıllı kontrat geliştirmenin araç takımı. Üç parçası var ve üçünü de kampta kullanacaksın.

Araç	Ne yapar
forge	Kontratı derler, test eder, ağa yayınlar.
cast	Zincirle konuşur. Bakiye sorar, işlem gönderir, kontrat okur.
anvil	Kendi bilgisayarında sahte bir blokzincir başlatır. İnternet gerekmez.

Kurulum

```
curl -L https://foundry.paradigm.xyz | bash
foundryup
forge --version
```

forge: command not found alırsan Foundry kurulmuş ama terminal onu bulamıyor. Şunu ekle:

```
export PATH="$HOME/.foundry/bin:$PATH"
```

Kalıcı olması için aynı satırı ~/.bashrc veya ~/.zshrc dosyasının sonuna yapıştır, terminali kapat aç.

İTERNETSİZ KALIRSAN

Foundry ilk derlemede Solidity derleyicisini internetten indiriyor. İndirdiği yer macOS'te `~/Library/Application Support/svm/`, Linux'ta `~/svm/`. Bağlantı kesikken taze bir bilgisayarda derleme yapamazsın. Kampa gelmeden bir kere `forge build` çalıştır ki derleyici inmiş olsun.

0.6 Cüzdan

Cüzdan aslında para tutmuyor. Para zincirde duruyor. Cüzdanın tuttuğu şey özel anahtar, yani o parayı hareket ettirme yetkisi.

Kampta MetaMask kullanacağız. Tarayıcı eklentisi olarak kur, kurulum sırasında sana on iki kelimelik bir kurtarma cümlesi verecek.

BU ÜÇ KURAL PAZARLIĞA AÇIK DEĞİL

1. Kurtarma cümlesini kağıda yaz. Ekran görüntüsü alma, buluta yükleme, kendine mesaj atma.
2. Kimse senden kurtarma cümlemini istemez. İsteyen dolandırıcıdır. İstisnası yok.
3. Kampta kullandığın cüzdanda gerçek para tutma. Testnet için ayrı bir cüzdan aç.

Base Sepolia ağını ekle

MetaMask varsayılan olarak sadece ana ağı gösterir. Test ağını elle eklemen gerekir:

Alan	Değer
Ağ adı	Base Sepolia
RPC adresi	https://sepolia.base.org
Zincir kimliği	84532
Sembol	ETH
Explorer	https://sepolia.basescan.org

Test parası

Testnet parasının değeri yok ama işlem yapmak için gerekiyor. Musluk denen sitelerden alınıyor. Uyarı: muslukların çoğu senden ana ağda gerçek para bulundurmanı istiyor. İstemeyen tek yer Coinbase Developer Platform musluğu, ücretsiz hesap yetiyor.

Rakamlar konusunda içini rahatlatalım. Base bir L2 olduğu için işlemler çok ucuz. Bu kampın tamamı, yani iki kontrat yayınlama artı yirmi mesaj artı yirmi transfer, tek bir öğrenci için yaklaşık 0.0000146 ETH tutuyor. Musluktan tek seferde aldığın miktar bunun birkaç katı.

0.7 Editör ve eklentiler

VS Code kur. WSL kullanıyorsan Remote WSL eklentisini de kur, yoksa Windows tarafındaki dosyaları düzenlemeye çalışırsın.

İşine yarayacak eklentiler:

- **Solidity** (Juan Blanco veya Nomic Foundation). Renklendirme ve hata gösterimi.
- **Remote WSL**. Sadece Windows için, ama şart.
- **GitLens**. Bir satırı kimin ne zaman değiştirdiğini gösterir.
- **Even Better TOML**. Foundry ayar dosyaları için.

Kampın ilk iki oturumunda VS Code kullanmıyoruz. Remix denen tarayıcı editörüyle çalışıyoruz, hiçbir kurulum istemiyor. VS Code ikinci günün öğleden sonrasında giriyor.

0.8 OpenZeppelin ve kütüphaneler

OpenZeppelin, akıllı kontratların standart kütüphanesi. Token, sahiplik, erişim kontrolü gibi herkesin ihtiyaç duyduğu şeyler orada yazılı ve defalarca denetlenmiş.

Neden kendin yazmıyorsun: ERC-20 standardını sıfırdan yazmak yüz elli satır sürer ve o yüz elli satırın içinde en az bir hata olur. Kampın üçüncü gününde tam olarak bunun örneğini göreceksin. Yapay zekanın yazdığı, derlenen, testleri geçen ve buna rağmen bir cüzdanı boşaltabilecek bir token kontratını inceleyeceğiz.

Denetlenmiş kütüphane kullanmak tembellik değil, güvenlik kararıdır. Bu cümleyi kampta beş kere duyacaksın.

Kurulum

Foundry projelerinde kütüphaneler lib/ klasörüne iniyor:

```
forge install OpenZeppelin/openzeppelin-contracts@v5.7.0
forge install foundry-rs/forge-std@v1.9.7
```

Sürümü yazmayı unutma. Yazmazsan en son sürüm iner ve kütüphane bir hafta sonra değiştiğinde kodun derlenmez.

remappings.txt

Solidity import satırlarının nereye baktığını bu dosya söylüyor. Yoksa import çözülmez:

```
forge-std/=lib/forge-std/src/
openzeppelin-contracts/=lib/openzeppelin-contracts/contracts/
```

0.9 İşine yarayacak diğer araçlar

Araç	Ne için
Remix (remix.ethereum.org)	Tarayıcıda Solidity yazmak. Kurulum yok. İlk kontratını burada yazacaksın.
Basescan	Zincirdeki her şeyi okumak. İşlem, kontrat, bakiye.
wagmi ve viem	Arayüzü zincire bağlayan JavaScript kütüphaneleri. Dördüncü gün.
Slither	Kontratı otomatik tarayıp bilinen hata kalıplarını bulur. Üçüncü gün deneyeceğiz.
Etherscan ABI	Doğrulanmış bir kontratın arayüz tanımını almak için.

Bunların hepsini şimdi kurmana gerek yok. Remix ve Basescan tarayıcıda çalışıyor, wagmi arayüz projesiyle birlikte geliyor.

0.10 Kurulum kontrol listesi

Kampın ikinci gününün öğleden sonrasına kadar bunların hepsi tamam olmalı:

- ☐ WSL kurulu ve Ubuntu terminali açılıyor (sadece Windows)
- ☐ `git --version` bir sayı döndürüyor
- ☐ GitHub hesabı var, SSH anahtarı çalışıyor
- ☐ `node --version` v20.9 veya üstü
- ☐ `forge --version` bir sayı döndürüyor
- ☐ Bir kere `forge build` çalıştırıldı, derleyici indi
- ☐ MetaMask kurulu, kurtarma cümlesi kağıtta
- ☐ Base Sepolia ağı MetaMask'e eklendi
- ☐ Cüzdanda test parası var
- ☐ VS Code kurulu, Solidity eklentisi var

BÖLÜM 1

Felsefe: Bu Şey Neden Var

Teknik kısma geçmeden önce bu sorunun cevabını bilmen gerekiyor. Çünkü blokzincir çoğu problemin çözümü değil ve bunu bilmeyen geliştirici, gerekmediği yerde kullanıp hem kendini hem kullanıcıyı yorar.

1.1 Merkeziyetsizlik neden doğdu

Basit bir soruyla başlayalım. Arkadaşına yüz lira borç verdin. Vermediğini söylerse bunu nasıl kanıtlarsın?

Verdiğin bütün cevapların ortak noktası var. Tanık, kağıt, mesaj kaydı, banka dekontu. Hepsinde bir üçüncü tarafa güveniyorsun. Tanığa, kağıdı saklayan kişiye, mesajı tutan şirkete, bankaya.

Hayatımız defter tutuculara emanet. Banka kimin ne kadar parası olduğunu biliyor. Tapu neyin kime ait olduğunu kaydediyor. Sosyal ağlar takipçini ve hesabını kendi sunucusunda tutuyor.

Bu sistem kötü değil. Bunu net söylemek lazım. Aracı olduğu için karşı tarafın dürüst olup olmadığını bilmek zorunda değilsin. Yüzyıllardır işliyor ve çoğu durumda gayet iyi işliyor.

Sorun aracının kendisi sorun olduğunda çıkıyor. Üç şekilde:

- 1. Tek nokta.** Defteri tutan çökerse, hacklenirse veya kapanırsa herkes etkilenir. Yedeği olsa bile kararı veren hala tek yer.
- 2. İzin.** Hesabın dondurulabilir, ödemen reddedilebilir, ülkeneye göre erişimin kesilebilir. Paran sende ama kullanma yetkisi başkasında.
- 3. Görünmezlik.** Defterin içeriğini göremezsin. Doğru tutulduğunu varsaymak zorundasın.

Merkeziyetsizlik bu üç sorunun cevabı olarak doğdu. Defteri tek bir kurum değil, herkes aynı anda tutsun fikri.

Ama bu fikri uygulamak için üç zor soruyu çözmek gerekiyor: Herkes kendi defterini tutuyorsa kim haklı? Biri defterine sahte bir satır yazarsa ne olur? Aynı parayı iki kişiye aynı anda gönderirsen ne olur?

Bu kitabın ikinci bölümü tam olarak bu üç sorunun cevabı.

Dağıtık ile merkeziyetsiz aynı şey değil

Bu ikisi sık karıştırılıyor, ayrımı bil.

Dağıtık sorusu şu: kaç kopya var? Defterin kopyası birçok makinede duruyorsa dağıtıktır. Bu teknik bir gerçek.

Merkeziyetsiz sorusu şu: kim karar veriyor? Kuralı değiştirebilecek tek bir taraf yoksa merkeziyetsizdir. Bu bir yetki gerçeği.

Dağıtık olup merkeziyetsiz olmayan sistem gayet mümkün. Beş banka ortak bir defter kurar, elli sunucuda elli kopya tutulur. Dağıtık, evet. Ama beşi bir araya gelip kuralı değiştirebiliyor, kaydı silebiliyorsa merkeziyetsiz değil.

Tersi olmaz. Dağıtmadan merkeziyetsiz olamazsın. Yani dağıtıklık gerekli ama yeterli değil.

1.2 Satoshi neden anonim kaldı

Bitcoin'i yazan kişi ya da grup 2008'de dokuz sayfalık bir makale yayınladı, birkaç yıl geliştirmeye devam etti, sonra ortadan kayboldu. Kim olduğu hala bilinmiyor.

Bu bir tarz meselesi değil. Sistemin çalışması için gerekliydi. Üç sebep:

Kurucu varsa baskı noktası vardır

Bir para sistemi düşün. Kurucusu belli, adresi belli. O kişiye baskı yapabilirsin. Tutuklayabilirsin, dava açabilirsin, tehdit edebilirsin, satın alabilirsin. Sistemin merkeziyetsiz olduğunu iddia edersin ama kurucusunun kapısı bir merkezdir.

Satoshi'nin kim olduğu bilinmediği için kapı yok. Bitcoin'i durdurmak isteyen birinin gidebileceği bir adres yok.

Kurucu varsa otorite vardır

Bir tartışma çıktığında herkes kurucuya bakar. "Satoshi ne diyor?" sorusu sorulduğu anda merkeziyetsizlik biter. Karar veren bir kişi olmuş olur.

Satoshi çekildiği için topluluk kendi kararını vermek zorunda kaldı. Bugün Bitcoin'de bir değişiklik yapılacaksa tartışılıyor, oylanıyor, madenciler ve düğümler kabul ediyor ya da etmiyor. Yavaş ve dağınık bir süreç, ama tam da bu yüzden kimsenin tek başına değiştiremeyeceği bir sistem.

Fikir kişiden büyük olsun diye

Bir teknoloji kurucusuyla anılıyorsa kaderi de ona bağlanır. Kurucu itibarını kaybederse teknoloji de kaybeder.

Satoshi ortada olmadığı için Bitcoin bir kişinin hikayesi değil, bir protokolün hikayesi oldu.

BURADAN ÇIKAN DERS

Merkeziyetsiz bir şey inşa ediyorsan kendine sor: bu sistem ben yarın kaybolsam çalışmaya devam eder mi? Cevap hayırsa merkeziyetsiz değil, sadece dağıtık.

1.3 Kuruma değil koda güvenmek

Burada bir yanlış anlama var, düzeltelim. Blokzincir "güven gerektirmez" değil. Güvenin yerini değiştirir.

Klasik sistemde şuna güveniyorsun: bankanın niyetine, çalışanlarının dürüstlüğüne, yöneticisinin bugün kötü bir karar almayacağına. Yani **insanlara ve kurumlara** güveniyorsun.

Blokzincirde şuna güveniyorsun: kodun yazdığı gibi çalışacağına, matematiğin doğru olduğuna, çoğunluğun kuralı değiştiremeyeceğine. Yani **doğrulanabilir bir mekanizmaya** güveniyorsun.

Farkı şurada: kurumun niyetini göremezsin, kodun içini görebilirsin. Kontrat doğrulanmışsa herkes okuyabilir. Ne söz verdiğini satır satır kontrol edebilirsin.

Ama bu takas bedava değil

Koda güvenmenin bedeli şu: kod yanlışsa geri dönüş yok.

Bankada yanlış transfer yaparsan arayıp iptal ettirebilirsin. Blokzincirde yanlış adrese gönderdiysen para gitti. Kontratında bir hata varsa o hata da kalıcı, çünkü kontrat yayınlandıktan sonra kodu değiştirilemiyor.

Yani "koda güveniyoruz" cümlesi aynı zamanda "kodu yazan kişinin sorumluluğu çok büyük" demek. Kampın üçüncü günü tam olarak bunun üstüne kurulu.

Kod kanunsa, kanunu kim yazıyor

Kampta çok duyacağın "kod kanundur" sözü kulağa hoş geliyor ama eksik. Kanunu birisi yazıyor ve o birisi hata yapabiliyor.

Gerçekte olan şu: kod, yazıldığı andaki niyetin donmuş halidir. Niyetin yanlışsa kod da yanlış olur ve kimse itiraz edemez.

Bu yüzden bu işte test yazmak bir alışkanlık değil, zorunluluk.

1.4 Gereksiz blokzincir

Bu başlığı kimse anlatmıyor, ben anlatıyorum. Çünkü bir teknolojinin nerede işe yaramadığını bilmiyorsan onu doğru yerde kullanamazsın.

Blokzincirin dört zayıflığı

1. **Yavaşdır.** Sıradan bir veritabanı binlerce kat hızlıdır. Blokzincir saniyede birkaç yüz işlem yaparken bir PostgreSQL sunucusu on binlerce yapar.
2. **Pahalıdır.** Yazdığın her satırın bir bedeli var. Veritabanına yazmak bedavaya yakınken zincire yazmak para istiyor.
3. **Gizli değildir.** Yazdığın her şey herkese açık. Kontrata şifre koyamazsın, gizli sayı saklayamazsın. Yeni başlayanların en pahalı hatası budur.
4. **Değiştirilemez.** Bu genelde avantaj sayılır ama bir yükümlülüktür de. Yanlış veri yazdıysan silemezsin.

Şu soruyu sor

Bir proje fikri geldiğinde üç soruyu sırayla sor. Üçüne birden evet diyemiyorsan blokzincire ihtiyacın yok.

1. Birden fazla taraf var mı ve bu taraflar birbirine güvenmiyor mu?
2. Aralarında hakemlik yapacak ortak bir kurum yok mu, ya da varsa ona güvenmek istemiyorlar mı?
3. Kaydın değiştirilemez olması gerçekten gerekli mi?

Örnekler

Fikir	Blokzincir gerekli mi	Neden
Okul yoklama sistemi	Hayır	Tek taraf var, okul. Kimse okula karşı kanıt üretmiyor. Excel yeter.
Şirket içi belge arşivi	Hayır	Güvenmediğin taraf yok. Yedekli bir veritabanı daha hızlı ve ucuz.
Sadakat puanı uygulaması	Genelde hayır	Puanı veren şirket zaten tek otorite. Zincire koymak sadece maliyet ekler.
Sınır ötesi ödeme	Evet	Taraflar farklı ülkede, ortak bir banka yok, aracı yavaş ve pahalı.
Ortak kasa yönetimi	Evet	Beş kişi para koyuyor, hiçbiri diğerine tam güvenmiyor, kuralın koda yazılması işi çözüyor.
Bir şeyin belli bir anda var olduğunu ispat	Evet	Zaman damgası kimsenin değiştiremeyeceği bir yerde duruyor.

HACKATHON UYARISI

Jürilerin en çok gördüğü ve en çok elediği proje türü, aslında bir veritabanının çözdüğü işi blokzincire taşıyan projedir. Fikrini sunarken "neden blokzincir" sorusunun cevabı ilk otuz saniyede net olmalı. Cevabın yoksa fikri değiştir.

BÖLÜM 2

Temeller: Defter, Mühür, Mutabakat

Bu bölüm kampın birinci gününün yazılı hali. Köy Defteri Oyunu'nda elle keşfettiğin şeylerin isimleri burada.

2.1 Ortak defter fikri

Blokszincir dediğimiz şey teknik olarak bir defterdir. İçinde kimin kime ne gönderdiği yazar. Tek farkı, bu defterin binlerce kopyası var ve hepsi aynı anda güncelleniyor.

Bir işlem yapmak şu demek: yüksek sesle ilan etmek. "Ali'den Ayşe'ye 3 altın." Herkes duyar, herkes kendi kopyasına yazar.

Bu basit kuralın getirdiği ilk sonuç şu: duymayan yazamaz. İşlemin geçerli olması için ağın büyük kısmının onu duymuş olması gerekiyor. Gizlice işlem yapamazsın.

Sahte satır neden tutmuyor

Diyelim biri kendi defterine kimsenin söylemediği bir satır ekledi. Kendine on altın gelmiş gibi yazdı.

Defterleri karşılaştırdığında o satır sadece bir kopyada var, diğer on dördünde yok. Üstelik satırda adı geçen gönderici "ben öyle bir şey demedim" diyor.

Çoğunluk kazanır. Buna **mutabakat** denir. Bir kişi on dört kişiyi yenemez.

Peki çoğunluğu ele geçirmek mümkün mü? Teorik olarak evet. Pratikte pahalı. Sistemi ayakta tutan şey de bu: yalan söylemek sana ödediğinden fazlasına mal oluyor.

2.2 Çift harcama

Dijital paranın en eski problemi bu. Bir dosyayı kopyalamak bedava. O zaman parayı da kopyalayamaz mısın?

Somut hali şöyle: on altının var. Sınıfın sol yarısına "Ayşe'ye on gönderdim" diyorsun, sağ yarısına aynı anda "Mehmet'e on gönderdim" diyorsun.

Sol yarı bir şey yazıyor, sağ yarı başka bir şey. İki farklı doğru ortaya çıkıyor. Ama ikisi birden olamaz, çünkü sadece on altının var.

Çözüm sıradır. Hangisi önce yazıldıysa o geçerli, diğeri düşer. Ve bu sıraya tek bir kişi karar veremez, ağ karar verir.

Bitcoin'in çözdüğü asıl problem buydu. Para göndermeyi zaten biliyorduk, aynı parayı iki kere gönderemediğinden emin olmayı bilmiyorduk.

2.3 Mühür ve değiştirilemezlik

Şimdiki soru şu: bugün yazılan doğru, peki ya biri yarın geçmişe dönüp eski bir sayfayı değiştirirse?

Çözüm zincirin adını aldığı yer. Her sayfa bir önceki sayfanın mührünü taşıyor.

Mühür, sayfanın içeriğinden hesaplanan bir sayı. Kampta elle yaptığımız basit hali şöyleydi:

```
muhur = sayfadaki altın miktarlarının toplamı + önceki sayfanın muhuru
```

Örnek:

```
Sayfa 1
Ali    -> Ayse    : 3
Mehmet -> Ali      : 5
Ayse   -> Can      : 2
önceki muhür: 0
muhur = 3 + 5 + 2 + 0 = 10

Sayfa 2
Can     -> Ali      : 4
Ali     -> Mehmet   : 6
Can     -> Ayse     : 1
önceki muhür: 10
muhur = 4 + 6 + 1 + 10 = 21
```

Şimdi biri sayfa 1'deki 3'ü 8 yapsın. Mührü yeniden hesapladığında 15 çıkıyor. Ama sayfa 2'de "önceki mühür: 10" yazıyor. Tutmuyor.

Yakalandı. Ve düzeltmek için sayfa 2'yi, sonra 3'ü, sonra hepsini değiştirmesi lazım. Hem de binlerce kopyada aynı anda.

Gerçek sistemlerde bu mührü **hash** deniyor ve toplama değil kriptografik bir fonksiyon kullanılıyor. Fikir aynı: içerik değişirse mühür değişir, mühür değişirse zincir kopar.

2.4 Cüzdan, adres, özel anahtar

Üç kavram, sık karışıyor.

Kavram	Nedir	Benzetme
Özel anahtar	Rastgele üretilmiş çok büyük bir sayı. Parayı hareket ettirme yetkisi.	İmza
Adres	Özel anahtardan matematiksel olarak türetilir. Herkese verebilirsin.	IBAN'ın
Cüzdan	Özel anahtarı saklayan ve senin adına imzalayan program.	Cüzdanın

Kritik nokta: adresten özel anahtara geri gidilemiyor. Tek yönlü. Bu yüzden adresini paylaşmak güvenli.

İkinci kritik nokta: özel anahtarını kaybedersen paranı kaybedersin. "Şifremi unuttum" diyebileceğin bir yer yok. Sistemin tamamı bunun üstüne kurulu, çünkü şifreni sıfırlayabilecek bir merkez olsaydı o merkez paranı da alabilirdi.

Kurtarma cümlesi

On iki kelime, özel anahtarın insan okuyabilir hali. Bir cüzdanı başka bir cihazda geri yüklemeye yarıyor.

Bu cümleyi bilen kişi paranın sahibidir. Başka doğrulama yok. Bu yüzden hiç kimseyle paylaşılmaz.

2.5 Bir işlemin yolculuğu

Gönder tuşuna bastığında ne oluyor. Beş adım.

1. İmzala

Cüzdanın işlemi özel anahtarınla imzalıyor. İmza, "bunu gerçekten ben istedim" demenin matematiksel yolu. Kimse taklit edemiyor ama herkes doğrulayabiliyor.

Dikkat: imzalarken özel anahtarın bilgisayarından çıkmıyor. İmza çıkıyor, anahtar değil.

2. Ağa duyur

İmzalı işlem binlerce düğüme aynı anda gönderiliyor.

3. Bekleme havuzu

İşlem hemen deftere yazılmıyor. Önce bir bekleme havuzuna giriyor. İngilizcesi mempool.

Havuz kalabalıkta daha çok ödeyen önce geçiyor. Kargodaki ekspres seçeneği gibi.

4. Bloğa gir

Bir düğüm havuzdan işlemleri toplayıp bir blok oluşturuyor ve zincire ekliyor. İşlemin ancak burada gerçekleşmiş sayılıyor.

5. Onaylandı

Blok zincire eklendikten sonra üstüne yeni bloklar geliyor. Her yeni blok işlemini biraz daha kalıcı yapıyor.

GERİ ALMA TUŞU YOK

Onaylanmış bir işlem iptal edilemez. Yanlış adrese gönderdiysen para gitti. Karşı taraf iyi niyetle geri göndermezse yapabileceğin bir şey yok.

Bu yüzden büyük transferlerden önce küçük bir test transferi yapmak yaygın bir alışkanlıktır.

2.6 Ücret neden var, L2 neden ucuz

Her bloğa sınırlı sayıda işlem sığıyor. Yer kıt olduğu için fiyatı var. Kalabalık saatte pahalı, sakin saatte ucuz.

Fiyatı kimse belirlemiyor, talep belirliyor. Sen sadece "acelem var mı" kararını veriyorsun.

L2 nedir

Ana ağda her işlem tek tek yazılıyor. Yer kıt, ücret yüksek.

L2 dediğimiz katman şunu yapıyor: binlerce işlemi kendi üstünde topluyor, hepsini tek bir zarfa koyuyor, zarfı ana ağa gönderiyor. Ana ağ tek bir işlem görüyor, ücreti içindekiler bölüşüyor.

Taksiyle gitmek yerine servis dolmuşuyla gitmek gibi. Kampta Base kullanıyoruz ve Base bir L2.

Rakamla görelim

Base Sepolia'da bu kampın tamamı, bir öğrenci için:

```
2 kontrat yayınlama + 10 mesaj yazma + 10 transfer  
= yaklaşık 2.400.000 gaz  
= 0,0000146 ETH
```

Ana ağda aynı iş yüzlerce kat pahalıya gelirdi.

BÖLÜM 3

Akıllı Kontratlar

Kampın ikinci günü. Buraya kadar defteri okuduk, şimdi deftere kural yazıyoruz.

3.1 Otomat benzetmesi

Otomata parayı atarsın, ürün düşer. Satıcıya güvenmene gerek yok. Pazarlık yok, "yarın gel" yok, keyfine göre fiyat değiştirmek yok. Makine kuralı neyse onu yapar.

Akıllı kontrat bu. Kuralı önceden yazarsın, sonra kimse fikrini değiştiremez. Sen bile.

Ne değil

- Hukuki sözleşme değil.** Mahkemeye gitmez, avukat okumaz. Sadece kod çalışır.
- Yapay zeka değil.** "Akıllı" ismi yanıltıcı. Düşünmüyor, sadece yazdığını yapıyor.
- Sihir değil.** Sen ne yazdıysan onu yapar. Yanlış yazdıysan yanlış yapar.

3.2 Üç parça: hafıza, işlem, duyuru

Her kontrat üç şeyden oluşuyor.

Parça	Solidity karşılığı	Ne işe yarar
Hafıza	state variable	Kontratın hatırladığı şeyler. Kim ne yazdı, kimin ne kadarı var.
İşlem	function	Dışarıdan çağrılabilen fonksiyonlar. Hafızayı değiştirir.
Duyuru	event	"Şu oldu" diye ağa bırakılan iz. Arayüz bunu okuyor.

En basit hali:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract SirinceDefteri {
    string public lastMessage;           // HAFIZA

    event MessageWritten(address author, string message); // DUYURU

    function write(string calldata message) external { // ISLEM
        lastMessage = message;
        emit MessageWritten(msg.sender, message);
    }
}
```

İlk iki satır her Solidity dosyasında olur. Birincisi lisans, ikincisi hangi derleyici sürümüyle çalışacağı.

public ne demek

Bir değişkeni `public` yaptığında Solidity otomatik olarak onu okumaya yarayan bir fonksiyon üretiyor. Yani `lastMessage` yazınca bedava bir okuma fonksiyonu kazanıyorsun.

Okumak bedava, yazmak ücretli

Hafızayı okumak zinciri değiştirmede için para istemiyor. Hafızayı değiştiren her işlem ücretli. Bu ayrımı dördüncü günde arayüzde yeniden göreceksin.

3.3 msg.sender ve kural yazmak

`msg.sender`, bu fonksiyonu şu an kim çağırdıysa onun adresi. Sabit değil, her çağrıda değişir.

Kural yazmak şu demek: `msg.sender`'a bakıp işleme izin vermek ya da vermemek.

```
if (msg.sender != owner) revert NotOwner();
```

"Çağırان sahip değilse işlemi iptal et." Kod kanundur dediğimiz şey tam olarak bu satır.

`revert` işlemi tamamen geri alıyor. Yarım kalmış bir değişiklik olmuyor. Ya hepsi olur ya hiçbiri.

3.4 Custom error, modifier, sahiplik

Sahiplik

Kontratı kim yayınladıysa sahibi odur. Bunu constructor'da kaydediyoruz. Constructor sadece bir kere, yayınlama anında çalışan fonksiyon.

```
address public immutable owner;

constructor() {
    owner = msg.sender;
}
```

`immutable`, bir kere yazılır bir daha değişmez demek. Sahip zaten değişmeyecekse bunu koda söylemek hem daha ucuz hem daha güvenli.

Custom error

Eski Solidity kodlarında `require(kosul, "hata mesajı")` görürsün. Bunun yerine custom error kullan:

```
error NotOwner();
error EmptyMessage();

function write(string calldata message) external {
    if (bytes(message).length == 0) revert EmptyMessage();
    ...
}
```

İki sebebi var. Metin mesajı zincire yazılıyor ve yer kaplıyor, yani para. İkincisi, hata bir tip olduğu için testte tam olarak hangi hatanın atıldığını kontrol edebiliyorsun.

Modifier

Aynı kontrolü iki fonksiyona yazman gerekseydi kopyalar mıydın? Kopyalama. Modifier, tekrar eden kuralı bir kere yazıp her fonksiyona takmaya yarıyor.

```
modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

function clear() external onlyOwner {
    lastMessage = "";
}
```

Alt çizgi, fonksiyonun asıl gövdesinin çalışacağı yeri gösteriyor. Önce kontrol, sonra gövde.

3.5 ERC-20 ve neden sıfırdan yazmıyoruz

ERC-20, token'ların uyduğu standart. Hangi fonksiyonların olacağını ve nasıl davranacağını tarif ediyor. Cüzdanlar ve borsalar bu standarda göre çalıştığı için token'ın standarda uyarsa her yerde görünüyor.

Standartı sıfırdan yazmak yaklaşık yüz elli satır sürer ve içinde en az bir hata olur. Onun yerine OpenZeppelin'den miras alıyoruz.

```
import {ERC20} from "openzeppelin-contracts/token/ERC20/ERC20.sol";
import {Ownable} from "openzeppelin-contracts/access/Ownable.sol";

contract SirinceToken is ERC20, Ownable {
    uint256 public immutable maxSupply;

    error CapExceeded(uint256 requested, uint256 remaining);

    constructor(string memory name_, string memory symbol_, uint256 maxSupply_)
        ERC20(name_, symbol_)
        Ownable(msg.sender)
    {
        maxSupply = maxSupply_;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        uint256 remaining = maxSupply - totalSupply();
        if (amount > remaining) revert CapExceeded(amount, remaining);
        _mint(to, amount);
    }
}
```

Otuz satır. Transfer, bakiye, onay, hepsi mirastan geliyor. Sen sadece kendi kuralını ekliyorsun: arz tavanı ve kimin basabileceği.

Ondalık meselesi

Solidity'de ondalıklı sayı yok. Token'lar bunu 18 basamak kaydırarak çözüyor. Yani bir token aslında 1.000.000.000.000.000.000 birim. Kodda 1e18 yazdığında bir token demiş oluyorsun.

3.6 Deploy ve kaynak kodu doğrulama

Deploy, kontratı zincire yazma işlemi. Bu andan sonra kod kilitleniyor.

Remix'ten

Environment olarak "Injected Provider" seç, cüzdanın Base Sepolia'da olsun, Deploy'a bas. Cüzdan onay isteyecek ve ücreti gösterecek.

Terminalden

```
cd contracts
cp ../.env.example .env      # PRIVATE_KEY satirini doldur
source .env
forge script script/Deploy.s.sol --rpc-url base_sepolia --broadcast --private-key
$PRIVATE_KEY
```

--broadcast yazmadan çalıştırırsan sadece simülasyon yapar, tahmini ücreti söyler, hiçbir şey harcamaz. Önce böyle dene.

.ENV NEREDE DURACAK

contracts/ klasörünün içinde, foundry.toml dosyasının yanında. Repo kökünde durursa Foundry onu göremiyor ve RPC_URL bulunamıyor. .env asla commit edilmez, .gitignore içinde olduğundan emin ol.

Kaynak kodu doğrulama

Deploy ettiğinde zincire giden şey senin yazdığın Solidity değil, derlenmiş makine kodu. Kimse okuyamaz. Doğrulama, kaynak kodunu explorer'a yükleyip "bu makine kodu bu kaynaktan çıktı" dedirtmek.

Doğrulanmamış kontrat, okunamayan sözleşme demek. Kimse ne yaptığını bilemez.

Bu adım en çok patlayan adımdır. Üç şey derleme anındaki ayarla birebir aynı olmalı:

1. Derleyici sürümü
2. Optimizer açık mı kapalı mı (Remix'te varsayılan kapalı)
3. Lisans satırı

Tutmazsa "bytecode does not match" alırsın. Ayarları kontrol et.

BÖLÜM 4

AI ile Kod Yazmak

2026'da bir geliştiriciye "yapay zeka kullanma" demek anlamsız. Herkes kullanıyor, şirketler de kullanıyor. Fark yaratan şey kullanıp kullanmaman değil, çıktısını değerlendirebilmen.

4.1 Prompt'un dört parçası

İyi bir istek dört parçadan oluşuyor. Biri eksikse çıktı bozuluyor ve genelde nerede bozulduğunu anlamıyorsun.

Bağlam

Ne üzerinde çalışıyoruz. Hangi proje, hangi dosya, hangi araçlar, hangi sürümler.

Yapay zeka senin ekranını görmüyor. Projenin Foundry mi Hardhat mı olduğunu, Solidity sürümünü, OpenZeppelin kullanıp kullanmadığını bilmiyor. Söylemezsen tahmin ediyor ve genelde en yaygın olanı seçiyor, ki o seninki olmayabilir.

Zayıf: "Bir token kontratı yaz."

İyi: "Foundry projesi, Solidity 0.8.28, OpenZeppelin v5 kurulu. contracts/src/ altında çalışıyorum, import yolu openzeppelin-contracts/ ile başlıyor."

Rol

Hangi gözle bakmasını istiyorsun. Aynı kodu yazan kişiyle denetleyen kişi farklı şeyler görür.

Rol vermek modelin bilgisini artırmıyor, dikkatini yönlendiriyor. "Kıdemli güvenlik denetçisi gibi bak" dediğinde farklı şeylere odaklanıyor.

Zayıf: "Bu kodu kontrol et."

İyi: "Bu kontratı denetleyen bir güvenlik uzmanı gibi oku. Amacın hata bulmak, kodu övmek değil. Bulamazsan bulamadığını söyle."

İçerik

Üzerinde çalışacağı malzeme. Kod, hata çıktısı, şartname, test.

En sık yapılan hata hata mesajını özetlemek. "Bir hata alıyorum" demek yerine hatanın tamamını yapıştır. İçindeki dosya adı ve satır numarası, senin gereksiz sandığın kısım, modelin en çok işine yarayan kısım.

Zayıf: "Test geçmiyor, düzeltir misin?"

İyi: "Şu test kırmızı. Test dosyasının tamamı şu. Kontratın tamamı şu. forge test çıktısı aynen şu."

Hedef

Ne olursa iş bitti diyeceğiz. Kabul kriteri.

Hedefi söylemezsen model kendi hedefini uyduruyor. Genelde "çalışan bir şey üretmek" diye anlıyor, oysa senin hedefin "mevcut testleri bozmadan şu davranışı eklemek" olabilir.

Zayıf: "Günlük limit ekle."

İyi: "Günlük transfer limiti ekle. Bitti sayacağımız yer şurası: forge test yeşil olacak, mevcut dört test bozulmayacak, limit her adres için ayrı sayılacak."

DÖRDÜNÜ BİRLEŞTİR

Foundry projesi, Solidity 0.8.28, OpenZeppelin v5. `contracts/src/SirinceToken.sol` dosyası ekte. *(bağlam)*

Test yazan bir geliştirici gibi davran, implementasyon yazma. *(rol)*

Kural şu: bir adres 24 saatte en fazla X token gönderebilir, limit her adres için ayrı sayılır, sahibi limiti değiştirebilir, limit aşılsa işlem reddedilir. *(içerik)*

Bana sadece Foundry test dosyası ver. Her testin ne kanıtladığını bir cümleyle yaz. *(hedef)*

4.2 AI ile kod yazmanın beş kuralı

- 1. Anlamadığın kodu deploy etme.** Kampın altın kuralı. Kontrat yayınlandıktan sonra geri dönüş yok ve sorumluluk sende.
- 2. Önce şartname, sonra test, en son kod.** Sırayı bozarsan doğrunun tanımını da yapay zekaya bırakmış olursun.
- 3. Çıktıyı satır satır oku.** Anlamadığın satırda durup sor. Geçme.
- 4. Testi değil kodu düzelttiğinden emin ol.** Bir test kırmızıysa modelin en kolay çözümü testi değiştirmektir. `git diff` ile kontrol et.
- 5. Kütüphane varsa kullan.** Yapay zekaya ERC-20 yazdırma. OpenZeppelin'den miras al.

4.3 Şartname, test, kod döngüsü

Doğru sıra şu ve bu sıra tesadüf değil.

1. Şartnameyi insan yazar

Kod yazma. Kuralı düz Türkçe yaz. Dört beş madde yeter.

Bu adımı yapay zekaya devretme. Çünkü şartname, ne istediğini bilmek demek ve ne istediğini bilmek senin işin.

2. Testleri yapay zeka yazar, insan onaylar

Prompt'ta açıkça söyle: sadece test istiyorum, implementasyon istemiyorum.

Çıkan her test için kendine sor: bu test ne kanıtlıyor? Cevabı yoksa test gereksizdir. Eksik gördüğün testi kendin ekle.

3. Testleri çalıştır, kırmızı olsun

Henüz kod yok, doğal olarak geçmeyecek.

Muhtemelen kırmızı test değil, derleme hatası göreceksin. Çünkü testler henüz var olmayan bir fonksiyonu çağırıyor ve proje derlenmiyor bile. Bu bir aksaklık değil, aynı dersin daha sert hali: testi yazdık, kod o kadar yok ki derlenmiyor.

4. Kodu yapay zeka yazar

Şimdi implementasyonu iste. Testlerin var olduğunu ve onları geçmesi gerektiğini söyle.

5. Testler karar verir

Geçmeyen varsa hata çıktısını olduğu gibi ver, düzelttir. Sonra tekrar oku.

6. Kritik soru

Bir test geçtiğinde kendine sor: kodu mu düzeltti, testi mi?

```
git diff
```

Testi değiştirdiyse ciddi bir sorun var. Doğrunun tanımını değiştirmiş demektir.

4.4 AI'nın yapamadıkları

Bunları bilmek, güvenebileceğin yeri bilmek demek.

- **Yeni mekanizma tasarlamak.** Daha önce yazılmamış bir ekonomik kurguyu kurmak onun işi değil. Var olan kalıpları birleştiriyor.
- **Saldıyı öngörmek.** Kodun ekonomik olarak nasıl sömürüleceğini genelde göremiyor. Sözdizimi hatasını bulur, teşvik hatasını bulmaz.
- **Güncel olmak.** Bu ayki protokol değişikliğini, kütüphanenin yeni sürümünü bilmeyebilir. Sürüm konusunda söylediğini doğrula.
- **Senin bağlamını bilmek.** Projenin geri kalanını, kullanıcılarını, kısıtlarını bilmiyor. Sen söylersen biliyor.
- **Emin olmamak.** Bilmediğinde de kendinden emin yazıyor. Bu en tehlikeli özelliği.

Araçlarla birlikte çalıştır

	Otomatik analiz araçları	Yapay zeka
Güçlü yanı	Kesin ve tekrarlanabilir	Geniş, bağlamı okuyor
Zayıf yanı	Dar, sadece bildiği kalıpları görür	Emin değil, uydurabilir
İyi olduğu iş	Bilinen zafiyet kalıpları	Araç çıktısını yorumlamak

İkisini birlikte kullan. Slither'ı çalıştır, çıktısını yapay zekaya yorumlat, sonuca kendin karar ver.

4.5 Bug'ı bulmak: yeşil test yalan söyler

Kampın üçüncü gününde bir kontrat inceleyeceksin. Yapay zekanın "basit bir token yaz, approve ve transferFrom olsun" isteğiyle yazdığı bir kontrat.

Derleniyor. Okunaklı. İki testi geçiyor.

Ve bir cüzdanı boşaltabilir.

Tuzak tam olarak burada. Testler yeşil olmak, kodun doğru olması demek değil. Yeşil test sadece şunu söyler: yazdığın testler geçiyor. Yazmadığın testler hakkında hiçbir şey söylemez.

Nasıl bakılır

- 1. Mevcut testleri çalıştır.** Yeşil. Bu yanlış bir güven duygusu üretiyor, farkına var.
- 2. Kodu gözle oku.** Yapay zeka kullanmadan. Her fonksiyonun ne söz verdiğini yaz. Sonra o sözü tutup tutmadığına bak.
- 3. Otomatik araç çalıştır.** Slither veya benzeri. Bulabilir de bulamayabilir de. İkisi de öğretici.
- 4. Bug'ı kanıtlayan testi yaz.** Test kırmızı olmalı. Kırmızı test, bug'ın varlığının kanıtıdır.
- 5. Düzelt.** Test yeşile dönsün, diğerleri hala geçsin.

Sorulacak soru genelde şu şekilde: bu fonksiyon ne garanti ediyor ve bir sonraki fonksiyon o garantiyi koruyor mu?

BURADAN ÇIKAN CÜMLE

Bu kontrat derleniyordu, okunaklıydı, testleri geçiyordu ve bir cüzdanı boşaltabilirdi. Bu yüzden ERC-20'yi sıfırdan yazmıyoruz.

BÖLÜM 5

Güvenlik

Kodun kalıcı. Hatan da kalıcı. Bu bölüm kampın en akılda kalan kırk beş dakikasının yazılı hali.

5.1 Reentrancy: bir kasa nasıl boşaltılır

Yirmi satırlık bir kasa kontratı düşün. İnsanlar para yatırıyor, istediklerinde çekiyor.

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "no balance");

    (bool ok, ) = msg.sender.call{value: amount}(""); // once gönderiyor
    require(ok, "transfer failed");

    balances[msg.sender] = 0; // sonra sıfırlıyor
}
```

Bu kodda gözle görülmeyen bir hata var. Çoğu insan ilk bakışta bulamıyor.

Sorun sırada. Kasa parayı gönderiyor, ama bakiyeyi sıfırlamayı sonraya bırakıyor.

Saldırı nasıl işliyor

Saldırgan sıradan bir kullanıcı değil, bir kontrat. Ve kontratların özel bir fonksiyonu var: para geldiğinde otomatik çalışan `receive()`.

```
receive() external payable {
    if (address(kasa).balance >= DEPOSIT) {
        kasa.withdraw(); // tekrar çağırıyor
    }
}
```

Adım adım:

1. Saldırgan `withdraw()` çağırır.
2. Kasa önce parayı gönderir.
3. Para gelince saldırmanın `receive()` fonksiyonu tetiklenir.
4. Oradan tekrar `withdraw()` çağrılır.
5. Kasa hala eski bakiyeye bakıyor, çünkü sıfırlama satırına henüz gelmedi. Tekrar gönderir.
6. İkinci adıma dön.

Döngü kasa boşalana kadar sürüyor.

Kampta canlı çalıştırdığımız test şunu gösteriyor: üç dürüst kullanıcı on beş ether yatırmış. Saldırgan bir ether ile geliyor, on altı ether ile çıkıyor.

ONCE	kasa	: 15 ether	saldırgan : 1 ether
SONRA	kasa	: 0 ether	saldırgan : 16 ether

5.2 Önce kontrol et, sonra kaydet, en son gönder

Yamanın tamamı bir satırın yerini değiştirmek.

```
function withdraw() external {
  // 1. KONTROL
  uint256 amount = balances[msg.sender];
  if (amount == 0) revert NoBalance();

  // 2. KAYDET (dis cagridan ONCE)
  balances[msg.sender] = 0;

  // 3. GONDER
  (bool ok, ) = msg.sender.call{value: amount}("");
  if (!ok) revert TransferFailed();
}
```

Artık saldırgan tekrar girdiğinde bakiyesi sıfır görünüyor ve işlem geri alınıyor.

Bu kalıbın adı Checks Effects Interactions. Türkçesi kampta tahtada duran cümle:

ÖNCE KONTROL ET, SONRA KAYDET, EN SON GÖNDER

Kural şu kadar basit: dışarıya bir çağrı yapmadan önce kendi hafızandaki işi bitir. Çünkü dışarı çağrı yaptığın anda kontrolü karşı tarafa veriyorsun ve o taraf sana geri dönebiliyor.

5.3 Üç gerçek vaka kalıbı

Gerçek dünyada tekrar tekrar aynı üç kalıp görülüyor.

Kapı açık kalmış

Belgede "bunu sadece sahibi çağırabilir" yazıyor ama koda yazılmamış. Erişim kontrolü unutulmuş. Herkes çağırılmış.

Önlemi: her fonksiyon için "bunu kim çağırabilir" sorusunu sor ve cevabı koda yaz.

Fiyata kanmış

Kontrat bir varlığın fiyatını tek bir kaynaktan okuyor. Saldırgan o kaynağı kısa süreliğine manipüle ediyor, kontrat yanlış fiyata göre işlem yapıyor.

Önlemi: tek kaynağa güvenme, birden fazla kaynak veya zaman ortalaması kullan.

Anahtar çalınmış

Kod kusursuz. Sorun geliştiricinin bilgisayarında. Özel anahtar bir yerde açıkta kalmış, bir dosyaya yazılmış, bir ekran görüntüsünde görünmüş.

Önlemi: anahtar asla kodda ve asla repoda durmaz. Ciddi para söz konusuysa donanım cüzdanı ve çoklu imza.

Bu üç kalıbın her biri için gerçek vakalar ve çalınan tutarlar var. Kampta güncel rakamlarla anlatılıyor.

Ortak ders

Güvenlik sadece kod değil. En zayıf halka genelde insan.

5.4 Yeni başlayanın kontrol listesi

Bir kontratı yayınlamadan önce bu beş soruyu cevapla. Fotoğrafını çek, her projede aç.

- 1. Bu fonksiyonu kim çağırabilir?** Cevabı koda yazdım mı, yoksa sadece kafamda mı?
- 2. Para hangi sırayla hareket ediyor?** Dışarı çağrıdan önce kendi kaydımı bitirdim mi?
- 3. Bir sayı beklediğimden büyük veya küçük olabilir mi?** Sıfır, negatif, tavan değeri denendi mi?
- 4. Anahtar kimde, nerede duruyor?** Repoda mı, ortam değişkeninde mi, donanımda mı?
- 5. Bu davranışı kanıtlayan bir testim var mı?** Yoksa davranış yok sayılır.

KAPANIŞ CÜMLESİ

Kodun değiştirilemez. Hatan da değiştirilemez.

BÖLÜM 6

Arayüz: Zinciri Ekranı Bağlamak

Kampın dördüncü günü. Yazdığın kontrat şu ana kadar sadece Remix'te ve explorer'da görünüyordu. Şimdi normal bir web sayfasına bağlıyoruz.

6.1 Okuma ve yazma farkı

Zincirle iki türlü konuşuluyor ve ikisi tamamen farklı.

	Okuma	Yazma
Ücret	Bedava	Ücretli
Süre	Anında	Beklemeli
Cüzdan	Gerekmez	Onay ister
Sonuç	Hemen döner	Blok bekler

Okuma bedava çünkü zinciri değiştirmiyor. Herhangi bir düğüme soruyorsun, cevabı veriyor.

Yazma bir işlem gönderiyor. İmza gerekiyor, ücret gerekiyor, blok beklemesi gerekiyor.

Bu fark arayüz tasarımının tamamını belirliyor. Okuma yapan bir bileşen sade olabilir. Yazma yapan bir bileşen üç durumu da göstermek zorunda: bekliyor, başarılı, hata.

6.2 wagmi ve viem

İki kütüphane, birlikte çalışıyorlar.

- **viem** alt katman. Zincirle konuşan, veriyi kodlayıp çözen kısım.
- **wagmi** üst katman. React için hazır kancalar sunuyor.

Yapılandırma

Hangi ağa bağlandığını ve nasıl bağlanacağını burada söylüyorsun:

```
import { http, createConfig, injected } from "wagmi";
import { baseSepolia } from "wagmi/chains";

export const config = createConfig({
  chains: [baseSepolia],
  connectors: [injected()],
  transports: { [baseSepolia.id]: http() },
});
```

injected()), tarayıcıda kurulu cüzdanı kullan demek. MetaMask gibi.

ABI nedir

Kontratın arayüz tanımı. Hangi fonksiyonlar var, hangi parametreleri alıyor, ne döndürüyor. Arayüzün kontratla konuşabilmesi için buna ihtiyacı var.

Tamamını koymana gerek yok, sadece kullandığın fonksiyonları koy.

Okuma

```
const { data: lastMessage } = useReadContract({
  address: DEFTER_ADDRESS,
  abi: DEFTER_ABI,
  functionName: "lastMessage",
});
```

Yazma

```
const { data: hash, writeContract, isPending, error } = useWriteContract();

writeContract({
  address: DEFTER_ADDRESS,
  abi: DEFTER_ABI,
  functionName: "write",
  args: [draft],
});
```

6.3 Bekleme ve hata durumu

Burası çoğu projenin eksik bıraktığı yer ve kullanıcı deneyimini belirleyen yer.

İmzalandı, onaylandı demek değil

Kullanıcı cüzdanda onay verdiğinde işlem gönderiliyor ama henüz zincire yazılmadı. İki ayrı durum var:

```
isPending    // cüzdan onayı bekleniyor
isConfirming // işlem gönderildi, blok bekleniyor
```

Butonun her ikisinde de farklı yazması gerekiyor. Aksi halde kullanıcı bir şey olmadığını sanıp tekrar basıyor ve iki işlem gönderiyor.

```
{isPending ? "Cüzdan onayı bekleniyor..."
: isConfirming ? "Zincire yazılıyor..."
: "Gonder"}
```

Üç hata yolunu da düşün

1. Kullanıcı cüzdanda reddedebilir.

2. Cüzdanında yeterli para olmayabilir.
3. Ağ düşebilir veya işlem geri alınabilir.

Üçünde de sayfa donmamalı, hata ekrana basılmalı. Kullanıcı ne olduğunu bilmeli.

İşlem geçtikten sonra veriyi tazele

İşlem onaylandığında ekrandaki veri hala eski. Yenilemen gerekiyor:

```
const { isSuccess } = useWaitForTransactionReceipt({ hash });

useEffect(() => {
  if (isSuccess) refetch();
}, [isSuccess]);
```

SIK YAPILAN HATA

Sayfa boş geliyor ve hata da vermiyor. Neredeyse her zaman iki sebepten biri. Önce kontrat adresini kontrol et, sonra cüzdanın hangi ağda olduğunu. Yanlış ağdaysan okuma sessizce boş döner.

BÖLÜM 7

Hackathon

Kampın son günü bir Demo Day ile bitiyor. Ama bu bölüm sadece kamp için değil. Bundan sonra gideceğin her hackathon için geçerli.

7.1 Üç dakikada sunum

Demo Day formatı: takım başına üç dakika. Bu süre sandığından kısa. Prova etmezsen ilk dakikada tükenirsin.

Süre	Ne anlatılır
İlk 30 saniye	Ne yaptınız. Tek cümlede problem, tek cümlede çözüm.
Sonraki 90 saniye	Canlı göster. Konuşma, tıkla.
Son 60 saniye	En zor kısım neydi, nasıl çözdünüz.

İlk otuz saniye her şeyi belirliyor

Jüri ilk yarım dakikada ne yaptığını anlamazsa geri kalanını dinlemez. Sunuma "biz düşündük ki" diye başlama. Şöyle başla:

"Bir grup arkadaş ortak bir kasaya para koyuyor ama kimse diğerine tam güvenmiyor. Biz bunu çözen bir kontrat yazdık. Üç kişiden ikisi onaylamadan para çıkmıyor."

İki cümle. Problem ve çözüm. Teknoloji adı bile geçmedi, gerek de yok.

Canlı göster, slayt gösterme

Doksan saniyelik demo bölümünde ekranda uygulaman olsun. Mimari şeması, teknoloji logoları, yol haritası slaytı gösterme. Jüri çalışan bir şey görmek istiyor.

Demo çökerse

Önceden ekran kaydı al. Canlı demo çökerse videoyu aç, sunuma devam et. Panikleyip ortamı düzeltmeye çalışma, süre işliyor.

7.2 Jüri neye bakar

Jüri üyeleri farklı şeylere değer verir ama neredeyse hepsi şu dört soruyu sorar:

- Çalışıyor mu?** Gerçekten çalışan bir şey mi, yoksa mockup mu. Bu en ağır kriter.
- Neden blokszincir?** Aynı işi bir veritabanı yapıyorsa proje zayıflar. Cevabın hazır olsun.
- Kim yaptı?** Takımın kodu anlıyor mu. Soru sorulduğunda cevap verebiliyor musunuz.

4. **Sonrası var mı?** Hackathon bitince devam edecek mi, yoksa bir hafta sonu projesi mi.

Dördüncü soruya "devam edeceğiz" demek yetmez. Neyi devam ettireceğini somut söyle.

7.3 En sık yapılan sekiz hata

1. **Kapsamı kesmemek.** Bitmeyen proje sunulamaz. Kural: tek kontrat, tek ekran, tek buton. Fazlası yok.
2. **Son saatte özellik eklemek.** Son iki saatte kod yazma. Prova et, ekran kaydı al, sunumu hazırla.
3. **Anlamadığı kodu kopyalamak.** Jüri "bu satır ne yapıyor" diye sorduğunda cevap veremezsin ve proje orada biter.
4. **Hiç test yazmamak.** Bir tane bile olsa yaz. "Testimiz var" demek ile dememek arasında ciddi fark var.
5. **Ana ağa deploy etmek.** Testnet yeterli. Ana ağa çıkmak ne etkileyici ne akıllıca.
6. **Özel anahtarı repoya koymak.** Her hackathonda birileri yapıyor. `.gitignore` dosyanı kontrol et.
7. **Sunumda teknoloji saymak.** Kullandığın kütüphaneleri listelemek kimseyi etkilemiyor. Ne yaptığını göster.
8. **Tek kişinin kodlaması.** Bir kişi kod yazıp diğerleri beklerse takım öğrenmiyor ve sunum çöküyor. İş böl.

7.4 Kampın ardından nasıl çalışılır

Bu dört gün bir başlangıç. Devamı gelmezse unutulur. Devamının nasıl geleceği şu:

Küçük ve bitmiş şeyler yap

Büyük bir proje başlatıp yarıda bırakmak, küçük bir projeyi bitirmekten daha az öğretiyor. Bitmiş bir şeyin bir sonu, bir demosu ve bir dersi oluyor.

İki haftada bitecek şeyler seç. Bitir. Sonra bir sonrakine geç.

Halka açık çalış

Yazdığın her şeyi GitHub'a koy. Kimse bakmasa bile koy. Bir yıl sonra geriye dönüp baktığında ilerlemeni göreceksin ve bir işe başvururken göstereceğin şey bu olacak.

Başkasının kodunu oku

Yazmaktan çok okumak öğretiyor. OpenZeppelin kontratlarını aç ve oku. Anlamadığın yeri not al, sonra araştı.

Hata mesajını okumayı öğren

Cevabın yarısı zaten hata mesajında yazıyor. Okumadan yapıştırma alışkanlığı, geliştirici olmanın önündeki en büyük engeldir.

Hackathon'a git

Bir hafta sonunda ekip kurar, bir şey yapar, sunarsın. Kazanmasan bile portföyün, ekip arkadaşların ve tanışıklığın kalır. Çoğu insan ilk işini oradan buluyor.

Takıldığında ne yapılır

1. Hata mesajını sonuna kadar oku.
2. Yapay zekaya bağlam ver. Kodu, hatayı ve ne beklediğini birlikte ver.
3. Sor. Topluluklar sandığından çok daha yardımsever.

BÖLÜM 8

Yazılımcı ve Geliştirici

Bu bölüm teknik değil. Ama kampın en uzun vadeli faydası burada.

8.1 İki farklı kavram

Günlük dilde ikisi aynı anlamda kullanılıyor. Değiller.

Yazılımcı, verilen bir işi koda çeviren kişidir. Şartname geliyor, o şartnameyi çalışan bir programa dönüştürüyor. Ölçüsü şudur: istenen şeyi ne kadar doğru ve ne kadar hızlı yazdı.

Geliştirici, neyin yazılması gerektiğine karar veren kişidir. Problemi tanımlıyor, çözümü tasarlıyor, neyin yapılmayacağına karar veriyor. Kod bu işin sadece bir parçası.

Aradaki fark bir yetenek hiyerarşisi değil. Farklı işler.

Ama son üç yılda bu iki işten birinin değeri hızla düştü, diğerininki yükseldi.

8.2 AI yazmayı ucuzlattığında geriye ne kalıyor

Şunu net söyleyelim. Bir şartnameyi koda çevirme işi artık ucuz. Çok ucuz. Beş yıl önce bir günlük iş olan şey bugün on dakika sürüyor.

Bu, yazma becerisinin değersiz olduğu anlamına gelmiyor. Ama tek başına yazma becerisinin kıt olmadığı anlamına geliyor. Kıt olmayan şey pahalı olmuyor.

Peki ne kıt kaldı

Doğru problemi seçmek. Yapay zeka sana ne yapman gerektiğini söylemiyor. Sorduğun soruya cevap veriyor. Yanlış soruyu mükemmel cevaplayan bir sistem hala yanlış.

Neyin yapılmayacağına karar vermek. Bir projeyi bitiren şey eklenen özellikler değil, eklenmeyenlerdir. Yapay zeka sana "bunu yapma" demiyor, istediğin her özelliği yazıyor.

Çıktıyı değerlendirmek. Kod üretmek ucuzladıkça, üretilen kodun doğru olup olmadığına karar verebilmek pahalılaşıyor. Kampın üçüncü günü tam olarak bunun üstüne kurulu. Derlenen, okunaklı, testleri geçen ve yine de bir cüzdanı boşaltabilen bir kontrat gördün.

Sorumluluk almak. Bir kontrat yayınlandığında ve para kaybedildiğinde kimse yapay zekayı sorumlu tutmuyor. İmzayı sen atıyorsun.

Bağlamı taşımak. Kullanıcının kim olduğunu, hangi kısıtların gerçek olduğunu, geçen ay neyin denenip başarısız olduğunu bilen sensin. Bu bilgi hiçbir prompt'a sığmıyor.

Yani

Yazma işi ucuzladı. Karar verme işi ucuzlamadı. Yapay zekanın yaptığı şey, işin ağırlığını yazmaktan karar vermeye kaydırmak oldu.

Bu iyi bir haber. Çünkü kod yazmayı öğrenmek yıllar alıyordu ve o kapı çoğu insana kapalıydı. Şimdi kapı açık. Ama içeride farklı bir beceri isteniyor.

8.3 Builder olmak

Builder, bir şeyi baştan sona kurup çalışır hale getiren kişidir. Fikirden başlar, kısıtları anlar, ne yapılacağına karar verir, yapar, insanların önüne koyar, geri bildirimine göre değiştirir.

Bu tanımın içinde kod yazmak var ama tamamı o değil.

Builder'ın ayakta kalmasının sebebi

Yapay zeka bir aracı otomatikleştiriyor, bir amacı değil. Sana çekiç veriyor, ne inşa edeceğini söylemiyor.

Ve inşa etmek, kod yazmaktan ibaret olmadığı için, kod yazmanın ucuzlaması builder'ı zayıflatmıyor. Güçlendiriyor. Tek başına bir kişi, beş yıl önce beş kişinin yapabileceği işi yapabiliyor.

Blokzincirin bu tabloda yeri

Bu alan builder'lara nadir bir şey sunuyor. İzin gerekmiyor.

Bir kontrat yayınlamak için kimseden onay almıyorsun. Şirket kurman, anlaşma imzalaman, bir platformun seni kabul etmesi gerekmiyor. Kodu yazıyorsun, ağa gönderiyorsun, çalışıyor.

Bu kampta dört günde yaptığın şey tam olarak buydu. Bir cüzdan kurdun, bir kontrat yazdın, kendi paranı çıkardın, arayüz bağladın ve hepsini kimseden izin almadan yaptın.

Nereden devam edersin

Bugün itibarıyla elinde şunlar var: çalışan bir geliştirme ortamı, zincire yayınlanmış kendi kontratın, testleri okuyabilme becerisi, bir hack'in nasıl işlediğini görmüş olmak ve yapay zekayı denetleyebilecek kadar temel.

Eksik olan tek şey tekrar. Bir sonraki projeyi başlat.

KAPANIŞ

Anlamadığın kodu deploy etme. Ama anlamak için denemeyi bırakma.

EKLER

Ek A: Komut Referansı

Foundry

```
forge build                # kontratlari derle
forge build --offline      # internete cikmadan derle
forge test                 # butun testleri calistir
forge test -vv             # console.log ciktilarini goster
forge test --match-test isim # tek bir testi calistir
forge test --match-contract X # tek bir test kontratini calistir
forge fmt                  # kodu bicimlendir
forge install kullanıcı/repo@surum
forge script script/Deploy.s.sol --rpc-url base_sepolia --broadcast
```

cast

```
cast chain-id --rpc-url base_sepolia
cast block-number --rpc-url base_sepolia
cast balance ADRES --rpc-url base_sepolia --ether
cast code ADRES --rpc-url base_sepolia
cast tx TX_HASH --rpc-url base_sepolia
cast call ADRES "lastMessage()(string)" --rpc-url base_sepolia
cast send ADRES "write(string)" "merhaba" --rpc-url base_sepolia --private-key $PRIVATE_KEY
cast send ADRES --value 0.01ether --rpc-url base_sepolia --private-key $PRIVATE_KEY
cast wallet address --private-key $PRIVATE_KEY
```

anvil (yerel zincir)

```
anvil                    # 127.0.0.1:8545, zincir kimligi 31337
anvil --host 0.0.0.0     # ag uzerinden erisilebilir yap

forge create src/SirinceDefteri.sol:SirinceDefteri \
  --rpc-url http://127.0.0.1:8545 \
  --unlocked --from 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 \
  --broadcast
```

Git

```
git clone --recurse-submodules ADRES
git status
git diff
git diff DOSYA
git add .
git commit -m "mesaj"
git push
git pull
git log --oneline
git checkout -- DOSYA          # bir dosyadaki degisikligi geri al
```

Node ve arayüz

```
nvm install --lts
nvm use --lts
npm install
npm run dev          # gelistirme sunucusu
npm run build        # uretim derlemesi
npm audit            # bilinen guvenlik aciklarini listele
```


Ek B: Sözlük

Terim	Anlamı
ABI	Kontratın arayüz tanımı. Hangi fonksiyonlar var, hangi parametreleri alıyor.
Adres	Zincirdeki hesap kimliği. 0x ile başlayan 42 karakter.
Blok	Bir arada işlenen işlem paketi. Zincirin bir halkası.
Çift harcama	Aynı parayı iki kere göndermeye çalışmak.
Deploy	Kontratı zincire yayınlama işlemi.
ERC-20	Token standardı. Cüzdanların token'ı tanınması bu sayede.
Event	Kontratın ağa bıraktığı iz. Arayüzler bunu okur.
EVM	Ethereum Virtual Machine. Kontratların çalıştığı ortam. Base dahil birçok ağ EVM uyumlu.
Gaz	İşlemin hesaplama maliyeti birimi. Ücret buradan hesaplanır.
Hash	Bir veriden üretilen sabit uzunluktaki özet. Kampta mühür dediğimiz şey.
immutable	Bir kere yazılıp bir daha değişmeyen değişken.
L2	Ana ağın üstünde çalışan, işlemleri toplu gönderen katman. Base bir L2.
Mempool	İşlemlerin bloğa girmeyi beklediği havuz.
Mint	Yeni token üretmek.
modifier	Fonksiyonlara takılan tekrar eden kontrol.
msg.sender	Fonksiyonu şu an çağıran adresin kendisi.
Musluk (faucet)	Test parası dağıtan site.
Nonce	Bir adresin kaçınıcı işlemi olduğunu gösteren sayaç.
Reentrancy	Bir fonksiyonun bitmeden tekrar çağrılmasıyla yapılan saldırı.
revert	İşlemi tamamen geri alma. Yarım değişiklik kalmaz.
RPC	Bir düğümle konuşmak için kullanılan adres ve protokol.
Testnet	Gerçek para kullanılmayan deneme ağı. Bizimki Base Sepolia.
Wei	ETH'nin en küçük birimi. 1 ETH = 10 üzeri 18 wei.

Ek C: Kaynaklar

Belgelendirme

- Solidity resmi belgeleri: docs.soliditylang.org
- Foundry Book: book.getfoundry.sh
- OpenZeppelin Contracts: docs.openzeppelin.com/contracts
- Base geliştirici belgeleri: docs.base.org
- wagmi: wagmi.sh
- viem: viem.sh

Araçlar

- Remix: remix.ethereum.org
- Base Sepolia explorer: sepolia.basescan.org
- Base musluk listesi: docs.base.org/chain/network-faucets
- Coinbase Developer Platform musluğu: portal.cdp.coinbase.com

Pratik yapmak için

- Oyunlaştırılmış zafiyet bulmacaları. Kontratı hackleyerek öğreniyorsun.
- Açık kaynak projelere küçük katkılar. İlk işi genelde bu getiriyor.
- Hackathonlar.

RlayHub Blokzincir Geliştiriciliği Kampı, Nesin Matematik Köyü, Şirince, Ağustos 2026.
Kampın kodu, ders dosyaları ve slaytları: github.com/mericcintosun/rlay-blockchain